



U-Map: A Reference Map for Safe Handling of Runtime Uncertainties

Nishanth Laxman¹(✉), Chee Hung Koo², and Peter Liggesmeyer¹

¹ Software Engineering: Dependability, Technische Universität Kaiserslautern, Kaiserslautern, Germany

{nishanth.laxman,liggesmeyer}@cs.uni-kl.de

² Corporate Sector Research and Advanced Engineering, Robert Bosch GmbH, Renningen, Germany

cheehung.koo@bosch.com

Abstract. “Uncertainty is certain” – a well-established fact that challenges design and engineering of dynamic systems. Cyber Physical Systems (CPSs) must function and perform tasks safely in real world contexts that might not be engineered specifically for them. These dynamic contexts are often accompanied by the pervasive presence of uncertainty. The dynamic nature of such systems as well as their ever-growing complexity further complicate safety assurance and require a paradigm shift towards more effective runtime safety assurance techniques. Many of the present runtime safety assurance techniques consider certain dynamic aspects of the system and its context, but not the uncertainty aspects completely. This paper presents results from an ongoing research to effectively handle runtime uncertainties in a model-based approach to assure system safety. In this paper, we propose a reference map called Uncertainty Map (U-Map) that can be used during system design to handle runtime uncertainties and apply it to a case study. The U-Map consists of an exhaustive set of possible uncertainties that are mapped to sets of potentially resulting hazards as well as possible runtime mitigation measures. It is intended to facilitate the identification of uncertainty-induced hazards during early design and contribute to the safe handling of runtime uncertainties.

Keywords: Safety · Runtime uncertainty · Uncertainty handling · Cyber physical systems · Hazard analysis · MBSA · Bayesian networks

1 Introduction

Embedded systems of today have evolved through time and are becoming more complex, dynamic, adaptive, well integrated into our daily life and supposedly

The work leading to this paper was partially funded by the German Federal Ministry of Education and Research under grant number 01IS16043 Collaborative Embedded Systems (CrEst).

© Springer Nature Switzerland AG 2020

M. Zeller and K. Höfig (Eds.): IMBSA 2020, LNCS 12297, pp. 227–241, 2020.

https://doi.org/10.1007/978-3-030-58920-2_15

smart. These dynamic intelligent systems are increasingly appealing for domains like manufacturing and production, healthcare, automobiles, energy, agriculture and avionics, most of which are safety-critical in nature. Traditional systems have a static relation to their surrounding environment and are engineered based on the computational myth that their operational context is predictable and fully specifiable. Modern-day embedded systems are composed of an increasing number of parts, abstraction layers and frameworks which are developed independently and might result in scenarios which are not completely understood or even fully specified a priori [11]. These embedded systems are envisioned to dynamically interact and collaborate with each other to achieve a common goal and are commonly called Cyber Physical System (CPS), Collaborative Embedded System (CES), System of System (SOS) or Multi Agent System (MAS). A CPS constitutes the system under consideration along with its environment (context) that is necessary for its functioning. This paper is written in the context of CPS.

Uncertainty in the context of systems engineering and development has gained increasing importance in recent years. Through rapid changes in system types and system boundaries, the associated uncertainty remains inevitable due to the fact that the physical world is inherently uncertain. *Uncertainty* as a term is very broadly used and is researched upon in domains like economics [23], robotics [29], artificial intelligence [18] and primarily in systems engineering [8, 28]. In the domain of systems engineering alone, there are varied definitions and understandings of the term *uncertainty*. One of the most suitable definitions for this paper is provided by Ramirez et al. [27], and they describe uncertainty as “*a system state of incomplete or inconsistent knowledge such that it is not possible for the system to know which of two or more alternative states/configurations/values is true*”. According to the authors, uncertainty can be introduced into the system at any point in time during its development. These uncertainties can propagate through different stages of development and manifest themselves at runtime (i.e. during system’s operation) if left unresolved [27]. At runtime, uncertainty can lead to component failures because of ambiguity, imprecision or inconsistency in information exchanges or due to uncertain contexts, further resulting in unforeseen behavior of the system. These uncertainties at runtime will be the prime focus of this research.

Uncertainty is often confused for the variability of the system and its context, because of the obscure differences between them. The differentiation between uncertainty and system variability is vital for the system development as well as safety assurance. Anderson and Hattis [1] provide some perception on how to differentiate uncertainty and variability in general. Koopman et al. [22] justifies embracing both uncertainty and variability together in real world scenarios. Additionally, Hoffman and Hammonds [15] provide some insights into the association between uncertainty and variability and the need to distinguish them. In reality, this differentiation is very subjective to the use case and human expertise involved in modeling sophisticated CPS. The presented approach in this paper is developed by considering the system variability as well, thus providing flexibility in adapting it to suit different use cases.

Traditional safety assurance techniques presume that the complete set of system specifications and configurations are available during system design. Such presumptions can no longer be applied to the development of CPS due to its unpredictability at runtime. This scenario demands development of different runtime safety assurance techniques. Most of the existing techniques do consider dynamic aspects of the system and its variability, but not uncertainties (runtime) explicitly. The runtime uncertainties, both the epistemic and the aleatory form [26], might put the system into a hazard state which in turn might result in an accident/harm. Early possible detection of such hazards can ensure appropriate measures being taken to avoid catastrophic events. The research on handling uncertainties is branched and primarily based on mitigation of specific categories of uncertainty (explained in Sect. 2). This paper presents a novel method based on the model-based approach, which can be used to consider relevant uncertainties and handle them together, encompassing different mitigation strategies, specific to the uncertainties. To facilitate this, a reference map (framework) called the Uncertainty Map (U-Map) is introduced which encapsulates all relevant uncertainties, early detection of uncertainty-induced safety concerns at the design stage and corresponding mitigation measures. The U-Map in turn can be used at runtime to monitor uncertainty manifestations and handle them accordingly based on its defined mitigation measures.

The presented approach in this paper is used to handle uncertainties from the view point of safety assurance and is structured as follows. Sect. 2 briefly summarizes relevant works from the literature. In Sect. 3, the U-Map as a reference map is introduced along with the methodology to develop it. The application of the U-Map is demonstrated with the help of a use case in Sect. 4. Section 5 concludes this paper and gives an overview of future works.

2 Related Work

In the following, an overview of different runtime uncertainty classifications is given, followed by state-of-the-art techniques for handling uncertainties. Since many of these techniques do not handle the uncertainties from a safety assurance perspective, an overview of various relevant runtime safety assurance techniques are also provided.

2.1 Runtime Uncertainty Classification

There is a multitude of research conducted in the field of uncertainty to identify and classify different possible uncertainties. These classifications lead to further research pathways to identify uncertainties with greater granularity and more specific to the use cases. These classifications are able to provide insights for our uncertainty handling approach to deal with aspects that might be neglected by the existing standards today. An extensive and well-defined classification promises an enhanced safety assurance.

An early inception of uncertainty classification that targets systems engineering was done by Walker et al. [33], where uncertainties are classified based on three dimensions: *location* of manifestation of uncertainty, *level* of uncertainty (variation in a spectrum between total unawareness and deterministic knowledge) and *nature* of uncertainty (whether the uncertainty is due to inherent randomness or associated with imperfect knowledge).

Ramirez et al. [27] provide a taxonomy with a well elicited list of uncertainties and have classified the sources of uncertainty based on the different stages of system development. According to the authors, runtime uncertainty predominantly occurs due to the interaction between systems and their contexts. For instance, runtime uncertainties caused by sensor failures and unpredictable context might lead to inaccuracy, ambiguity or inconsistency, in which incomplete information available to the system might result in incorrectly interpreted events. Apart from the taxonomy, the authors also provide information regarding mitigation techniques to solve certain specific uncertainties.

The research on uncertainties for Self Adaptive System (SAS) that possess self-adaptation capabilities recognizes further new challenges and paves way for a separate branch of research [7]. With the focus on SAS, Perez-Palacin and Mirandola [26] present a taxonomy with more granular definitions based on the three similar categories provided by [33]. The authors characterize the uncertainty to be either epistemic or aleatory, based on its nature. An epistemic uncertainty is the result of the imperfection in the knowledge acquired. Possible reasons that cause epistemic uncertainty can be, a lack of information that would be required to create a reliable knowledge base, inaccurate measurements or models due to the negligence of certain aspects of context, or imperfection in processing the information. An aleatory uncertainty is the result of the inherent randomness of the system behavior and its operating context. The proposed approach in this paper takes both epistemic and aleatory uncertainty into account.

An extensive and sophisticated classification for sources of uncertainty in the context of SAS is also provided by Mahdavi-Hezavehi et al. [24]. Cámara et al. [5] further extend the classification of [24] by combining it with the classification provided by [27]. Both these studies, [24] and [5], classify the uncertainties in five dimensions: *location* where the uncertainty emerges, different *sources* of uncertainty, *nature* of uncertainty, *level/spectrum* as to on what scale the uncertainty is specified in (statistical or scenario based) and *emerging time* as to in which stage of system development, the uncertainty will emerge (manifest). These classifications are the most comprehensive ones available in the literature, which helps to identify a myriad of uncertainty possibilities and scenarios.

Hildebrandt et al. [14] also provide an uncertainty classification for epistemic uncertainties that deals with runtime information exchange between systems. The authors classify the communication-induced epistemic uncertainties into four type-level and four instance-level uncertainties based on a simplified knowledge modeling procedure using the ontology based approach [32]. These classification schemes serve as a checklist to conduct a systematic analysis on potential uncertainties.

2.2 Uncertainty Handling Techniques

Brun et al. [4] explain how uncertainty can be handled during the development of SAS with the help of feedback loops. The Monitor-Analyze-Plan-Execute-Knowledge (MAPE-K) feedback loops introduced by Kephart and Chess [19] take prime importance in their work. The MAPE-K feedback loops are envisioned to be one of the most prominent reference control models for autonomous systems. Runtime models are used to represent a system and its surrounding, are built for specific purposes and are expected to provide an acceptable level of accuracy and precision [3]. Giese et al. [13] investigate as to how runtime models help cope with uncertainty. They propose an extension to the existing MAPE-K feedback loop by combining them with the runtime models to handle uncertainty. Garlan et al. [12] propose a so-called Rainbow approach, using an architecture-based self-adaptation technique with reusable infrastructure and software architectures to support self-adaptation of software systems. This approach is also primarily based on the MAPE-K feedback loops. The proposed U-Map in this paper can also be integrated into the MAPE-K feedback loop for the runtime aspects of uncertainty handling.

Esfahani et al. [10] introduce a quantitative approach called POISED to determine the consequences of uncertainty. This approach aids in making adaptation strategies that will result in the best possible behavior considering the specific use case. Elkhodary et al. [9] propose FUSION, an approach based on probability theory that uses machine learning capabilities, and includes feature orientation and dynamic optimization to tune the adaptation behavior of the system in presence of uncertainty. Cámara et al. [6] contribute a formal framework to reason about the human participation in self-adaptation, particularly during its execution stage. Their work illustrates how explicit modeling of humans as participants (effectors) can provide a better insight into the involvement of humans for adaptation.

2.3 Runtime Safety Assurance Techniques

One of the well-known concepts to deal with runtime safety is proposed by Schneider et al. [30] that applies Conditional Safety Certificates (ConSerts) to certify open adaptive systems at runtime. ConSerts evaluate runtime evidences/preconditions to provide the necessary guarantees for safe system collaborations. Other relevant approaches for runtime safety analysis in the automotive domain are proposed by Östberg and Bengtsson [25] and by Kabir et al. [17] that focus mainly on cooperative automotive systems. To enable a better dependability analysis and assurance throughout the lifetime of a system, a concept using Digital Dependability Identities (DDIs) is also proposed by Schneider et al. [31] that utilize digital artifacts to describe dependability-related information. In the context of reconfigurable production systems, Koo et al. [21] propose a method using analysis modules called SAM-RT to allow runtime analysis and monitoring of safety aspects. These runtime safety assurance techniques can be integrated with the U-Map proposed in this paper.

3 The Uncertainty Map (U-Map)

The Uncertainty Map (U-Map) is a reference map, which acts as a knowledge base and also as guidelines for handling uncertainties at runtime. The U-Map consists of well identified possible list of uncertainties, mapped explicitly to different possible hazards which in turn are appropriately mapped to their corresponding mitigation measures.

Saffiotti [29] summarizes three attitudes towards problems involving uncertainty:

1. Removing the uncertainty: by using better hardware, sophisticated and high precision sensors and carefully engineering every aspect of the system and its environment.
2. Tolerating the uncertainty: by accepting the presence of uncertainty and using robust techniques to operate in wide range of situations and to recover from them.
3. Reasoning about the uncertainty: by explicit representation and manipulation of uncertain information and reasoning about its effects.

The first attitude involves high cost and does not cater to all runtime uncertainties, specifically not to the aleatory form of uncertainty. This attitude is more suitable for simple systems where all possible interactions can be modeled. As the CPS of our concern are highly complex and safety critical, the presented approach is a combination of both second and third attitudes to handle uncertainty, taking the tolerating attitude and enriching it by reasoning about the uncertainty.

The development of U-Map is done during system design and implemented during system integration. Here we introduce a three-fold mitigation strategy to yield its flexibility and applicability for wide range of use cases. The mitigation measures are split into three steps, undertaken either sequentially or independently. The three steps of mitigation measures are:

- * Rectification measure
- * Degradation measure
- * Additional protective measure

Rectification measure is the step taken as an attempt to eliminate some uncertainty during the integration of systems at runtime. There is some quotient of possibility associated with this step, and the steps taken are highly use case specific. It is already established in previous sections that eliminating uncertainty completely is not possible, however in this step uncertainty with respect to missing information and ambiguous information during information exchange at runtime is reduced (sometimes eliminated). The uncertainties that can be mitigated through rectification measure are generally associated with information exchange. Degradation measure refers to change in the functionality or configuration of the system to prevent hazardous event and ensure safety. This step involves runtime adaptation through reconfiguration. Provision for additional

protective measures is available when a critical state is reached and hazards cannot be prevented through rectification or degradation. These measures help reduce the severity of the accident which is most likely to occur. The mitigation steps taken are highly dependent on the type of uncertainty and level of uncertainty being considered. Few uncertainties can be completely eliminated during rectification, few might still lead to certain hazards after rectification and hence have to be mitigated through degradation measures or additional protective measures or both. Few other uncertainties might have to be mitigated independently and this very flexibility is offered by U-Map. Figure 1 shows the visualization of U-Map.

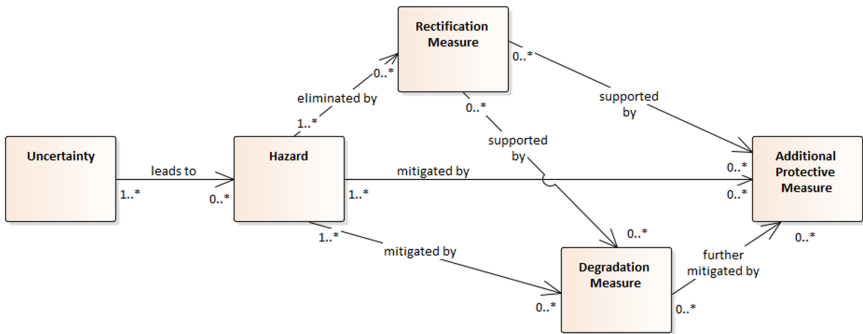


Fig. 1. The Uncertainty Map (U-Map)

In the following sections, we will focus on how the U-Map can be developed, its design and runtime aspects, as well as an exemplary case study to demonstrate its application.

3.1 Methodology to Develop the U-Map at design time

The development of U-Map is done in parallel to the system design after the initial scope of the system is defined. Once developed, the U-Map is implemented by providing suitable channels and interfaces for monitoring of the system and its context, and for execution of mitigation measures. The following are the important steps involved in creation of U-Map:

1. **Uncertainty identification:** This is a crucial and important step for runtime uncertainty handling and it is highly use case specific. This is done during development of CPS, so that appropriate channels are available at runtime for smooth integration of systems. This step involves listing out all possible runtime uncertainties based on the scope of the use case. To facilitate this step we make use of the uncertainty classifications explained in Sect. 2.1 from the research community, to look into the aspects of system interactions and behaviors that are generally not considered in the system development. The most sophisticated and extensive

classification by [5] is used in our approach to list out all relevant uncertainties that might manifest. Additionally, to identify uncertainties specifically with respect to the information exchange at runtime, classification by [14] is used. Both these classifications serve as a check list to identify possible uncertainties.

First the relevant categories from the classification are identified, following which individual instances of possible uncertainty manifestations are listed for every category specific to the use case. They are further iterated with granular specifications. These granular specifications serve as the parameters for runtime monitoring.

2. **Uncertainty-oriented hazard analysis:** Uncertainties generally do not manifest in a sequence or independently from each other, but rather together and often with different levels of uncertainty. This combination of uncertainty manifestations over time, affects the frequency of occurrence, controllability and severity of hazards.

This step involves a reasoning-based identification of hazards for every possible uncertainties (and combinations of different uncertainties). This step considers standardized hazards to alleviate further efforts in defining appropriate mitigation steps to prevent hazards. In the presented approach we use Bayesian Networks (BNs) [2], based on probability theory [20] to find the probability of occurrence of a hazardous event based on identified uncertainty manifestation. Probability is a measure of likelihood of an event occurring, and is usually quantified as a number in interval $[0, 1]$, where 0 represents impossibility and 1 represents certainty [20]. Through this approach we can see how probable a hazardous event will occur if the identified uncertainty manifests at runtime. BNs use Directed Acyclic Graphs (DAG) [2] for graphical representation of probabilistic models. In this paper, we use this as a representative method, which will be further explained in the context of a specific use case. However, detailed probabilistic modeling based on BNs is out of the scope for this paper.

3. **Mitigation planning:** This step involves defining appropriate counter measures to prevent and mitigate hazards, which requires a certain amount of planning along with reasoning. This mitigation planning involves checking whether it is feasible to reduce the uncertainty through rectification at runtime, and to ensure that corresponding channels and interfaces are available to do it. As explained in previous section, some of the uncertainties that can be rectified might still raise the question on whether the associated hazard can be eliminated completely. As some cases of uncertainty manifestations cannot be rectified completely through system design, they have to be mitigated through degradation measure, additional protective measure or the combination of both these measures at runtime. Here, we consider the mitigation steps recommended by existing standards and characterize them based on the three-fold mitigation technique, as defined in our approach.
4. **Consolidation of U-Map:** This step involves physical creation of the U-Map by combining the individual outputs from all the previous steps. In this step, all the identified uncertainties and their combinations are mapped to

corresponding hazards, which in turn to their mitigation steps to create a complete reference map architecture to handle uncertainties at runtime.

3.2 Runtime Aspects of the U-Map

The U-Map can be either integrated into the MAPE-K feedback loop architecture as the knowledge base and used as a standalone method to handle uncertainty, or be included into the existing runtime safety analysis techniques presented in Sect. 2.3. Alternatively, they can also be integrated into other general system architectures that have the capability of runtime monitoring and execution of mitigation measures during operation. At runtime, system and its context has to be continuously monitored for uncertainty manifestations. The uncertainty manifestations are then analysed based on the available information in U-Map to identify possible hazards and to execute corresponding mitigation measures to ensure safety. Depending on the use case, the runtime uncertainty analysis can be done when the system is in operation or during the planned downtime for reconfiguration. For both these scenarios, rectification measures can be defined accordingly. After the successful rectification, the likelihood of a particular hazardous event is expected to be reduced. Otherwise, other measures as described in U-Map will be used to handle the residual uncertainty during operation. The detailed explanation for the runtime uncertainty handling using U-Map will be provided in future publications.

4 Case Study

The application of the presented concept will be demonstrated using a use case based on an adaptable production scenario. Current trends in the production domain are moving towards the usage of automated and mobilized industrial robots. A Movable Robot (MR) is usually installed on a movable platform (e.g. Automated Guided Vehicle (AGV)) to guarantee its maximum production flexibility. It can be envisaged that such MR can be integrated instantaneously into a new production environment to perform a completely new production task. Through the existing infrastructure for Machine-to-machine (M2M) communication and process orchestration, a newly integrated MR receives concrete production instructions from the Orchestration Unit (OU), identifies actual configuration in the environment and performs the production tasks.

As shown in Fig. 2, a MR is integrated into a new Production Environment (PE) to perform a Production Goal (PG). PG is used as an umbrella term to explain all interactions of the MR with the rest of the production system. M2M communication and the task allocation can be made possible, thanks to the OU. We also consider Human Task Factor (HT) for the overall production context. The integration and setup of a new MR in the PE is done as follows:

1. MR communicates with the OU to provide information w.r.t its services, available degradation modes, runtime localization and its hazard zones.

2. OU considers the overall production process and assign tasks to MR, which includes working coordinates of the PG along with the motion planning.
3. MR might provide details regarding inherent safety devices (e.g. safety laser scanners) to monitor the surrounding.

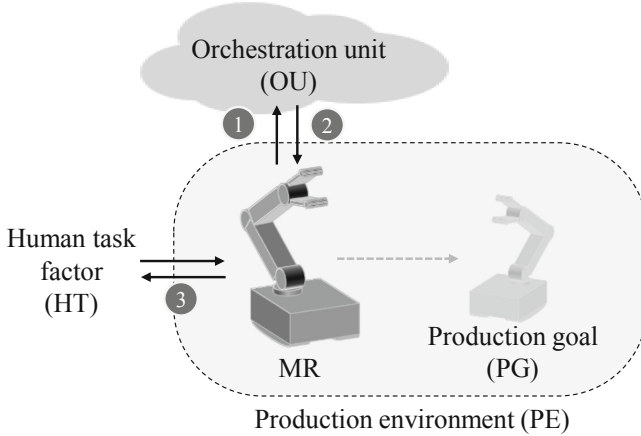


Fig. 2. The industrial scenario involving the integration of a Movable Robot (MR) into a new Production Environment (PE)

As the concrete MR to be integrated is usually unknown at design time, different uncertain situations might manifest during runtime integration which will lead to the emergence of safety concerns. In this paper, to ease the understanding we consider only few uncertainties from the perspective of one newly introduced MR. The first step involves identifying the relevant possible uncertainties (**UnC**) by using the classification from research community as a check list, described in Sect. 2.1. The following uncertainties were derived:

- **UnC1:** Unintended entering of human into the hazard zone (Aleatory type).
- **UnC2:** Mismatch in the hazard zone description defined by MR and OU because of different ontologies used for their development. (Epistemic type).
- **UnC3:** Unavailable or failure of safety devices.

These identified uncertainties, can be traced back to various root causes considering the MR's interactions with the environment. Especially in terms of data communication with the OU, a high amount of uncertainties might manifest during runtime integration. For the identified uncertainties, following possible hazards (**H**), based on the safety standard ISO 12100:2010 [16] are considered:

- **H1:** Impact of moving robot arm with the human.
- **H2:** New crushing point due to the formation of new hazard zone after integration of MR.
- **H3:** Shearing, cutting or damage to the work piece.
- **H4:** Trapping of human in the hazard zone.

The probability of occurrence of the hazards based on uncertainty manifestation probabilities is modeled using BNs as shown in Fig. 3 (The probability values for manifestation of uncertainties are only representative. In actual practice, they are retrieved based on frequentistic or subjectivistic methods). Here, the uncertainty **UnC1** manifests with a frequency of low (1–5 times per hour), med (5–10 times per hour) or high (10–15 times per hour) based on number of times there is unintended entering of human into the hazard zone. **UnC2** is based on whether there is a mismatch in the hazard zone definitions between MR and OU depending on respective ontology during integration. **UnC3** shows whether a particular inherent safety device is available or not. Initially, we can consider that the availability of safety device is high when the MR is being integrated, but it might be unavailable at runtime due to potential failure. In our BN, we have also identified the likelihood of the hazard resulting in an accident (instantaneously) or a delayed accident (after a certain time) or no accident and mapped individual probabilities of uncertainty manifestation to them.

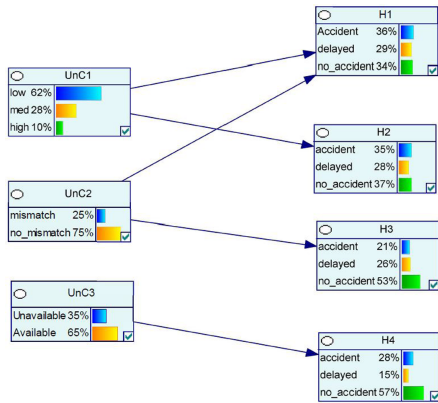


Fig. 3. Bayesian network model showing probability of occurrence of identified hazards based on probability of uncertainty manifestation

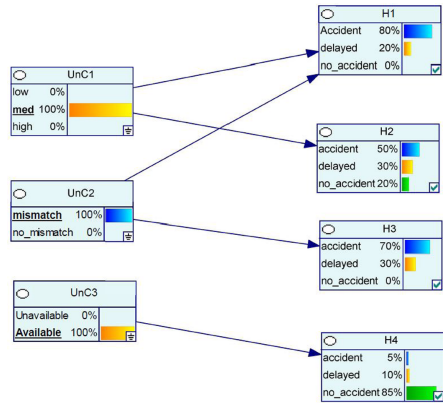


Fig. 4. Bayesian network model with evidence of uncertainty manifestation at runtime

A possible scenario is considered: a “med” frequency (5–10 times per hour) of unintended entering of human into hazard zone (**UnC1**) detected by the sensors, a mismatch in hazard zone definitions (**UnC2**), and availability of the inherent safety device. With these evidences, changes in the probability of occurrence of the identified hazards can be seen in the Fig. 4. Once the probability of occurrence of different hazards is known based on these BNs, the necessary mitigation measures can be defined. The following mitigation measures are identified:

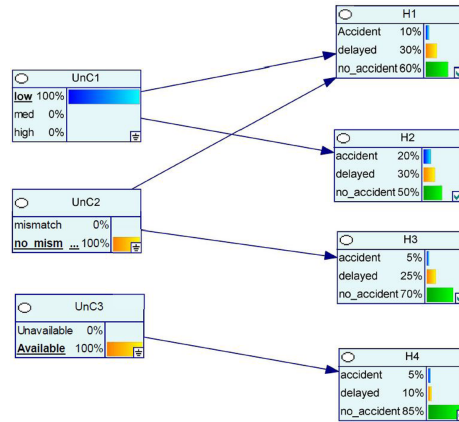


Fig. 5. Bayesian network model after rectification (R1, R2 and R3)

- **R1:** Rectification measure - Manual addition of the hazard zone description of MR to the OU by the safety engineer during certification to eliminate **UnC2**.
- **R2:** Rectification measure - Specific training on “Go/No Go” zones for the HT after integration of MR to reduce **UnC1**.
- **R3:** Addition of specific sensors to monitor the newly formed hazard zone after integration of MR to monitor both **UnC1** and **UnC2** manifestations.
- **D1:** Degradation measure - Reduction in speed of robot arm.
- **D2:** Degradation measure - Immediate STOP function.
- **D3:** Degradation measure - Slow retraction of robot arm to a default safe orientation.
- **AP1:** Additional protective measure - Raise Alarm (light/sound).
- **AP2:** Additional protective measure - Usage of attenuation screens or guards.

The rectification measures (**R1**, **R2** and **R3**) were appropriately undertaken, thereby completely eliminating **UnC2**, and reducing the probability of manifestation of **UnC1**. Corresponding degradation measures and additional protective measures (because of 35% probability of unavailability of inherent safety devices for **UnC3**) were also installed. It can be noted that the probability of occurrence of hazards (predominantly accidents) has greatly reduced, as shown in Fig. 5.

The complete visualisation of the resulting U-Map for the use case is shown in Fig. 6. It can be noticed that even after rectification, some uncertainty regarding randomness of human behavior will be present and can be mitigated by additional protective measures. For some hazards (**H1** and **H2**), both rectification and degradation measures are to be undertaken and one particular hazard (**H4**) can be mitigated only by additional protective measure.

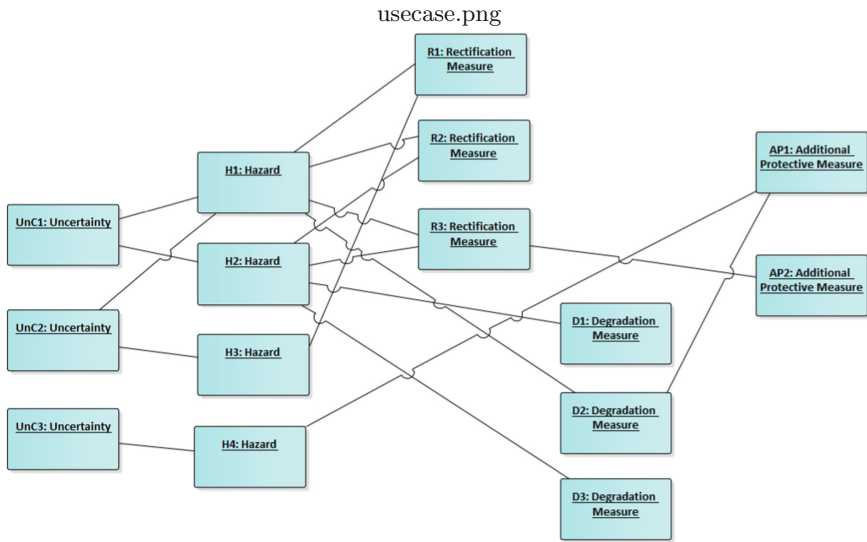


Fig. 6. U-Map for the use case

5 Conclusion and Future Work

Uncertainty pertaining to CPS is one constant aspect that cannot be completely eliminated. These uncertainties constantly pose safety-related issues and make it very crucial for safety critical systems. Ideally, through the operating life of the system, the level of uncertainty should slowly decrease over time, as different decisions are undertaken at runtime and likewise more information is being available to the system. However, this might be the exact opposite in reality. Therefore it is very important and necessary to address this problem by defining strategies to safely handle the uncertainties at runtime.

The proposed U-Map in this paper contributes well and serves as a reference map to handle runtime uncertainties in a highly flexible and scalable manner to suit different use cases. It can be used as a reference document in use cases where manual approach with safety engineers is used for safety assurance. In automated systems, it can be used as a knowledge base in MAPE-K feedback loops to analyse the uncertainty and execute identified mitigation steps.

Currently, the information on uncertainty sources and classifications is mainly based on existing literature, which might be incomplete for the proposed U-Map. A separate safety relevant taxonomy for runtime uncertainties should be investigated into and created in the future. Other future works involve the creation of U-Map for other domains, consideration of the temporal aspects of uncertainty manifestations while modeling BNs, defining monitor specifications for runtime monitoring of systems while considering scenario analysis and the identification of appropriate means to integrate the U-Map into the safety life cycle. The possibility of automating the creation of U-Map, also represents one of the interesting and challenging research directions in the future.

References

1. Anderson, E.L., Hattis, D.: A. uncertainty and variability. *Risk Analysis* **19**(1), 47–49 (1999)
2. Ben-Gal, I.: Bayesian networks. In: *Encyclopedia of Statistics in Quality and Reliability*, vol. 1. Springer, Dordrecht (2008). https://doi.org/10.1007/978-1-4020-6754-9_1624
3. Blair, G., Bencomo, N., France, R.B.: Models@ run. time. *Computer* **42**(10), 22–27 (2009)
4. Brun, Y., et al.: Engineering self-adaptive systems through feedback loops. In: Cheng, B.H.C., de Lemos, R., Giese, H., Inverardi, P., Magee, J. (eds.) *Software Engineering for Self-Adaptive Systems*. LNCS, vol. 5525, pp. 48–70. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02161-9_3
5. Cámara, J., Garlan, D., Kang, W.G., Peng, W., Schmerl, B.: Uncertainty in self-adaptive systems: Categories, management, and perspectives. Carnegie-Mellon Univ Pittsburg PA United States, Technical report (2017)
6. Cámara, J., Moreno, G., Garlan, D.: Reasoning about human participation in self-adaptive systems. In: *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, pp. 146–156. IEEE (2015)
7. Cheng, B.H.C., et al.: Software engineering for self-adaptive systems: a research roadmap. In: Cheng, B.H.C., de Lemos, R., Giese, H., Inverardi, P., Magee, J. (eds.) *Software Engineering for Self-Adaptive Systems*. LNCS, vol. 5525, pp. 1–26. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02161-9_1
8. Cheng, S.W., Garlan, D.: Handling uncertainty in autonomic systems. In: *International Workshop on Living with Uncertainty* (2007)
9. Elkhodary, A., Esfahani, N., Malek, S.: Fusion: a framework for engineering self-tuning self-adaptive software systems. In: *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pp. 7–16 (2010)
10. Esfahani, N., Kouroshfar, E., Malek, S.: Taming uncertainty in self-adaptive software. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, pp. 234–244 (2011)
11. Garlan, D.: Software engineering in an uncertain world. In: *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research*, pp. 125–128 (2010)
12. Garlan, D., Cheng, S.W., Huang, A.C., Schmerl, B., Steenkiste, P.: Rainbow: architecture-based self-adaptation with reusable infrastructure. *Computer* **37**(10), 46–54 (2004)
13. Giese, H., et al.: Living with uncertainty in the age of runtime models. In: Bencomo, N., France, R., Cheng, B.H.C., Aßmann, U. (eds.) *Models@run.time*. LNCS, vol. 8378, pp. 47–100. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-08915-7_3
14. Hildebrandt, C., Bandyszak, T., Petrovska, A., Laxman, N., Cioroica, E., Törsleff, S.: EURECA: epistemic uncertainty classification scheme for runtime information exchange in collaborative system groups. *SICS Software-Intens. Cyber-Phys. Syst.* **34**(4), 177–190 (2019)
15. Hoffman, F.O., Hammonds, J.S.: Propagation of uncertainty in risk assessments: the need to distinguish between uncertainty due to lack of knowledge and uncertainty due to variability. *Risk Anal.* **14**(5), 707–712 (1994)

16. International Organization for Standardization (ISO): ISO 12100: Safety of machinery-General principles for design-Risk assessment and risk reduction (2010)
17. Kabir, S., et al.: A runtime safety analysis concept for open adaptive systems. In: Papadopoulos, Y., Aslansefat, K., Katsaros, P., Bozzano, M. (eds.) IMBSA 2019. LNCS, vol. 11842, pp. 332–346. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-32872-6_22
18. Kanal, L.N., Lemmer, J.F.: Uncertainty in Artificial Intelligence. Elsevier, Amsterdam (2014)
19. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. *Computer* **36**(1), 41–50 (2003)
20. Kolmogorov, A.N., Bharucha-Reid, A.T.: Foundations of the Theory of Probability: Second, English edn. Courier Dover Publications, Mineola (2018)
21. Koo, C.H., Laxman, N., Möhrle, F.: Runtime safety analysis for reconfigurable production systems. In: The 30th European Safety and Reliability Conference (ESREL). Research Publishing, Singapore (2020, in press)
22. Koopman, P., Osyk, B., Weast, J.: Autonomous vehicles meet the physical world: RSS, variability, uncertainty, and proving safety. In: Romanovsky, A., Troubitsyna, E., Bitsch, F. (eds.) SAFECOMP 2019. LNCS, vol. 11698, pp. 245–253. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26601-1_17
23. Laffont, J.J.: Economie de l'incertain et de l'information. MIT Press, Cambridge (1989)
24. Mahdavi-Hezavehi, S., Avgeriou, P., Weyns, D.: A classification framework of uncertainty in architecture-based self-adaptive systems with multiple quality requirements. In: Managing Trade-Offs in Adaptable Software Architectures, pp. 45–77. Elsevier (2017)
25. Östberg, K., Bengtsson, M.: Run time safety analysis for automotive systems in an open and adaptive environment (2013)
26. Perez-Palacin, D., Mirandola, R.: Uncertainties in the modeling of self-adaptive systems: a taxonomy and an example of availability evaluation. In: Proceedings of the 5th ACM/SPEC International Conference on Performance Engineering, pp. 3–14 (2014)
27. Ramirez, A.J., Jensen, A.C., Cheng, B.H.: A taxonomy of uncertainty for dynamically adaptive systems. In: 2012 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 99–108. IEEE (2012)
28. Rowe, W.D.: Understanding uncertainty. *Risk Anal.* **14**(5), 743–750 (1994)
29. Saffiotti, A.: Handling uncertainty in control of autonomous robots. In: Hunter, A., Parsons, S. (eds.) Applications of Uncertainty Formalisms. LNCS (LNAI), vol. 1455, pp. 198–224. Springer, Heidelberg (1998). https://doi.org/10.1007/3-540-49426-X_10
30. Schneider, D., Becker, M., Trapp, M.: Approaching runtime trust assurance in open adaptive systems. In: Proceedings of the 6th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 196–201. Association for Computing Machinery, New York (2011)
31. Schneider, D., Trapp, M., Papadopoulos, Y., Armengaud, E., Zeller, M., Höfig, K.: WAP: digital dependability identities. In: 2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE), pp. 324–329. IEEE (2015)
32. Staab, S., Studer, R.: Handbook on Ontologies. Springer, Heidelberg (2010). <https://doi.org/10.1007/978-3-540-92673-3>
33. Walker, W.E., et al.: Defining uncertainty: a conceptual basis for uncertainty management in model-based decision support. *Integrat. Assess.* **4**(1), 5–17 (2003)